

OEF AI-Native Software Engineer Exercise

This exercise is for applicants to the AI-Native Software Engineer role at Open Earth Foundation. It helps us assess your technical skill, how you work with AI coding tools, and how you make decisions under time constraints.

We anticipate about **4 hours** of work on this project. If you get to the 4-hour point, please wrap up and let us know what else you would do.

Problem: City Climate Action Tracker

Cities around the world are committing to reduce their greenhouse gas emissions, but they need simple tools to track their climate actions and understand their progress toward reduction targets. A city government wants a lightweight app where their climate team can manage actions and the public can follow progress.

What to Build

Create a small web application with two roles:

City Admin (the climate team):

- Set a city's **baseline emissions** (total annual CO2 in tons) and a **target year** to reach net zero
- **Add, edit, and remove** climate actions. Each action has:
 - Title (e.g., "Expand bike lane network")
 - Sector (transport, energy, buildings, waste, land use)
 - Estimated annual CO2 reduction (tons/year)
 - Status (planned, in progress, completed)
 - Start year
- **Import an action from free text:** paste a paragraph describing a climate initiative (e.g., from a policy document or meeting notes), and an LLM extracts it into a structured action (title, sector, estimated reduction, status, start year). The admin can review and edit before saving.

Public Viewer (citizens):

- View the city's climate actions and current progress
- See a **summary dashboard** showing:
 - Total estimated reductions vs. baseline
 - A simple breakdown by sector

- Whether the city is on track to meet its target (green) or not (red)

We provide sample data below.

The frontend should be a **web application** - no need for mobile. We care more about a functional, well-structured UI than visual polish. A clean layout that makes the data easy to understand is more valuable than custom styling. You can use any tech stack you're comfortable with - we use Next.js, React, TypeScript, and PostgreSQL at OEF, but this is not a requirement for the exercise. For the LLM integration, you can use any API (OpenAI, Anthropic, etc.) or a local model.

Stretch goals (only if you have time):

- A chart or graph showing projected emissions per year
- Persistence (database or local storage)
- Ability to manage multiple cities
- Authentication for the admin role

Sample Data

JSON

```
{
  "city": "Greenville",
  "baselineEmissions": 500000,
  "targetYear": 2035,
  "actions": [
    {
      "title": "Expand bike lane network",
      "sector": "transport",
      "annualReduction": 12000,
      "status": "in progress",
      "startYear": 2024
    },
    {
      "title": "Solar panel incentive program",
      "sector": "energy",
      "annualReduction": 45000,
      "status": "in progress",
      "startYear": 2023
    },
    {
```

```

    "title": "Municipal building retrofits",
    "sector": "buildings",
    "annualReduction": 18000,
    "status": "planned",
    "startYear": 2026
  },
  {
    "title": "Organic waste composting program",
    "sector": "waste",
    "annualReduction": 8000,
    "status": "completed",
    "startYear": 2022
  },
  {
    "title": "Urban reforestation initiative",
    "sector": "land use",
    "annualReduction": 15000,
    "status": "planned",
    "startYear": 2025
  },
  {
    "title": "EV fleet transition for public transit",
    "sector": "transport",
    "annualReduction": 30000,
    "status": "planned",
    "startYear": 2026
  }
]
}

```

Example Free Text for Import

"The city council approved a \$2M investment to convert all street lighting to LED by 2027. The energy department estimates this will cut approximately 9,500 tons of CO2 per year once fully deployed. The project is currently in the planning phase."

Expected parsed output: Title: "LED street lighting conversion", Sector: energy, Annual reduction: 9500, Status: planned, Start year: 2027.

How You Build It Matters

This role requires daily use of AI coding tools. We want to see how you actually work with them.

Use AI coding tools (Cursor, Claude Code, GitHub Copilot, Codex, Windsurf, or any tool you prefer) while building this project.

Along with your code, include a short write-up (**1 page max, can be a markdown file in your repo**) covering:

1. **Which AI tools you used** and your general workflow with them
2. **One moment where AI saved you significant time** - what did you prompt, what did it generate, why was it good?
3. **One moment where you overrode or corrected the AI** - what did it get wrong, how did you catch it, what did you do instead?
4. **How you structured your session** - did you give the AI context up front? Did you use rules/instructions files? How did you break down the work?

We are not looking for a polished essay. Bullet points are fine. We want to understand your workflow and judgment.

Deliverables

- Code for your solution (GitHub repo or zip)
- Instructions for building and running it
- The AI workflow write-up described above
- A 30-minute Q&A session with our team to present your solution, walk through your AI workflow, discuss challenges, and talk about what you'd improve